

Summary

Comprehensive Briefing: Relational Operators, Logical Operators, and Expression Evaluation in C

This briefing presents a consolidated explanation of relational operators, logical operators, and expression evaluation in C programming. It also highlights the functional behavior of the `printf()` function when used inside expressions. The session emphasizes how C represents truth values numerically and how operator priority influences expression outcomes.

Key Takeaways:

- C does not use a native Boolean type in this context; instead:
 - **True = 1**
 - **False = 0**
- Any non-zero value is treated as True.
- Only zero is treated as False.
- Relational and logical operators always evaluate to 0 or 1.
- `printf()` returns the number of characters printed.
- Unary operators have the highest priority, while assignment has the lowest.

1. Relational Operators

Relational operators compare two operands and return a numeric result based on the comparison outcome.

1.1 Types and Evaluation

There are **six relational operators** in C. Each expression evaluates to:

- **1** if the condition is true
- **0** if the condition is false

Operator	Description	Example	Evaluation	Result
<	Less than	10 < 20	True	1

>	Greater than	10 > 90	False	0
<=	Less than or equal to	10 <= 10	True	1
>=	Greater than or equal to	10 >= 5	True	1
==	Equal to	10 == 10	True	1
!=	Not equal to	10 != 3	True	1

1.2 Priority and Associativity

Relational operators follow a defined hierarchy:

- **Higher Priority:** <, <=, >, >=
- **Lower Priority:** ==, !=
- **Associativity:** Left to Right

2. Logical Operators

Logical operators are used to combine relational expressions or reverse truth values.

2.1 Truth Representation in C

- **True** → Any non-zero value (e.g., 9, -17, 13.2)
- **False** → Only 0 or 0.0

2.2 Types of Logical Operators

1. Logical AND (&&)

- Returns 1 if both operands are non-zero
- Returns 0 otherwise

Examples:

9 && 13.2 → 1

17 && 0 → 0

2. Logical OR (||)

- Returns 1 if at least one operand is non-zero
- Returns 0 only if both operands are zero

Examples:

`13 || 2` → 1

`0 || 0.0` → 0

3. Logical NOT (!)

- Unary operator that reverses the truth value

Rules:

`!(non-zero)` → 0

`!(0)` → 1

3. The `printf()` Function in Expressions

In C, `printf()` is a **function**, not just an output command.

3.1 Return Value of `printf()`

- `printf()` returns the **number of characters printed**.

Examples:

```
i = printf("GATE");
```

- Printed characters: `G A T E`
- Return value: `4`
- Final value of `i`: `4`

```
printf("GATE 2026");
```

- Characters printed: `4 letters + 1 space + 4 digits`
- Return value: `9`

3.2 Nested `printf()` Evaluation

Expression:

```
i = printf("%d", printf("GATE 2026"));
```

Step-by-step Evaluation:

1. Inner `printf("GATE 2026")` executes first
 - Output: `GATE 2026`
 - Return value: `9`

Outer call becomes:

```
printf("%d", 9);
```

- 2.
3. Outer `printf` prints `9`
4. Characters printed by outer `printf`: `1`
5. Return value of outer `printf`: `1`
6. Value assigned to `i`: `1`

Final Output on Screen:

GATE 20269

Final Value of `i`:

1

4. Operator Priority Hierarchy

C evaluates expressions using a strict priority order (highest to lowest):

1. **Unary Operators**
Example: `!`
2. **Arithmetic (Multiplicative)**
`*, /, %`
3. **Arithmetic (Additive)**
`+, -`
4. **Relational (Comparison)**
`<, <=, >, >=`
5. **Relational (Equality)**
`==, !=`
6. **Logical Operators**
`&&, ||`
7. **Assignment Operator**
`=`

4.1 Associativity Rule

- Operators of the same priority are generally evaluated from Left to Right.
- Assignment associativity is an exception, but details were not included in the source data.

Final Insight

This briefing reinforces that accurate understanding of truth values, operator precedence, and function return behavior is essential for correctly evaluating expressions in C. These concepts are especially critical for exam problem solving and avoiding logical errors in real-world programs.

